

课程笔记

CS224N Lecture 10: Post-training - RLHF, SFT, DPO

整理：AI Course Notes Contributors；授课：Chris Manning

2024 年

课程版本：2024 年中文讲义整理版

笔记来源：AI Course Notes Contributors

本版处理：移除课件截图和对应图注，不包含 Stanford 原始课件、图片或视频。

授权：笔记依 CC BY-NC-SA 4.0 分享；完整许可与改动记录见下一页。

来源与许可

本中文笔记由 AI Course Notes Contributors 整理，课程版本为 2024；源稿：<https://github.com/hqhq1025/ai-course-notes/tree/717e2df65c3d2eee593c171156a30d4f8f6812b9/cs224n/lecture10>。

笔记内容依 CC BY-NC-SA 4.0 分享：<https://creativecommons.org/licenses/by-nc-sa/4.0/>。

本副本的改动：移除 Stanford 原始课件截图与依赖这些截图的图注，移除课程视频封面/链接，并清理 TeX 源稿中的行尾空白后重新编译为可独立阅读的 PDF；同时加入本来源与许可说明。完整许可文本随本文件夹提供。Stanford 课件、课件图片和视频不包含在该授权内；请从对应年份 Stanford 课程页访问原始课件。

目录

1 引言：从预训练到 ChatGPT	3
1.1 预训练学到了什么？	3
1.2 本章小结	3
2 Zero-shot 与 Few-shot In-context Learning	4
2.1 GPT 系列的演进	4
2.2 Zero-shot Learning	4
2.3 Few-shot In-context Learning	4
2.4 Chain-of-Thought 提示	4
2.5 In-context Learning 的局限性	5
2.6 本章小结	5
3 指令微调 (Supervised Fine-Tuning)	5
3.1 从预训练到指令遵循	5
3.2 规模效应：更大的模型从指令微调中获益更多	6
3.3 评估：MMLU 基准	6
3.4 指令微调的实用经验	6
3.5 指令微调的三大局限	7
3.6 本章小结	7
4 基于人类反馈的强化学习 (RLHF)	7
4.1 核心动机：直接优化人类偏好	7
4.2 问题形式化	7
4.3 Step 2: 训练奖励模型	8
4.3.1 为什么需要奖励模型？	8
4.3.2 为什么用排序而非评分？	8
4.3.3 Bradley-Terry 模型	8
4.4 Step 3: RL 优化与 KL 约束	9
4.5 RLHF 的惊人效果	9
4.6 RLHF 的复杂性	10
4.7 本章小结	10
5 直接偏好优化 (DPO)	10
5.1 核心思想：用语言模型自身表达奖励	10
5.2 数学推导	11
5.2.1 Step 1: KL 约束优化的闭式解	11
5.2.2 Step 2: 用语言模型表达奖励	11
5.2.3 Step 3: 配分函数的消除	11
5.2.4 最终的 DPO 损失函数	12
5.3 DPO 的实验效果	12
5.4 DPO 的广泛采用	12
5.5 SFT vs RLHF/DPO 的输出对比	12

5.6	本章小结	12
6	从 InstructGPT 到 ChatGPT	13
6.1	InstructGPT: 定义管线	13
6.2	ChatGPT: 对话优化	13
6.3	本章小结	13
7	开放挑战与未来方向	13
7.1	Reward Hacking	13
7.2	冗长性问题 (Verbosity)	13
7.3	幻觉问题	14
7.4	本章小结	14
8	总结与延伸	14
8.1	讲者的核心总结	14
8.2	全课知识图谱	14
8.3	关键 Takeaways	15
8.4	拓展阅读	15

1 引言：从预训练到 ChatGPT

本节课由斯坦福大学博士生 Archit Sharma 主讲，主题是大语言模型的**后训练** (Post-training)。课程的核心问题是：我们如何从一个仅仅学习预测下一个 token 的预训练模型，一步步走到像 ChatGPT 这样能够理解用户意图、遵循指令、生成高质量回答的强大助手？

课程路线图

本课覆盖四个核心主题，层层递进：

1. **Zero-shot 与 Few-shot In-context Learning**: 无需微调，通过提示词引导模型完成任务
2. **指令微调 (Instruction Fine-tuning / SFT)**: 收集指令-回答对，训练模型遵循指令
3. **基于人类偏好的优化 (RLHF)**: 引入人类偏好数据，通过强化学习对齐模型行为
4. **直接偏好优化 (DPO)**: 绕过强化学习，直接用偏好数据优化语言模型

1.1 预训练学到了什么？

预训练的规模正在指数级增长。2022 年模型大约使用 1.4 万亿 token 进行训练，而到 2024 年，Llama 3 已经使用了约 **15 万亿 token**。模型消耗的计算量也从 10^{24} FLOP 增长到超过 10^{26} FLOP。

尽管预训练的损失函数极其简单——仅仅是预测下一个 token，但模型在这个过程中学到的远不止语法和知识：

- **世界知识**：“斯坦福大学位于加利福尼亚州”
- **数学能力**：理解方程和图形
- **代码编写**：Copilot 等代码补全工具
- **Agent 心智模型**：能够根据角色背景预测不同人物的行为
- **医学诊断**：提供初步的症状分析（但不应作为医疗建议）

预训练不只是记忆

一个有趣的实验：当文本描述“Pat 是一名物理学家”时，模型预测保龄球和树叶会同时落地（正确的物理知识）；而当描述改为“Pat 从未见过这个实验”时，模型预测保龄球先落地（符合直觉但物理上不正确）。这说明模型不仅记住了事实，还学会了根据**角色背景**进行推理——这种能力远超简单的文本预测。

语言模型正在发展成为**通用多任务助手**。从简单的文本预测出发，它们已经能够处理代码、数学、创意写作、医疗咨询等多种任务。今天的课程将解释如何通过后训练技术进一步释放和对齐这些能力。

1.2 本章小结

预训练通过海量数据上的 next-token prediction 为模型注入了丰富的知识和推理能力。然而，预训练模型本质上是一个文本补全器，并非一个助手。后训练的目标就是将这个强大但“不听话”的补

全器，转变为一个能理解用户意图、遵循指令、安全可靠的 AI 助手。

2 Zero-shot 与 Few-shot In-context Learning

2.1 GPT 系列的演进

GPT 系列是后训练研究的重要背景。让我们回顾其演进：

模型	参数量	训练数据	关键突破
GPT-1 (2018)	117M	4.6 GB	预训练 + 微调范式
GPT-2 (2019)	1.5B	40 GB	Zero-shot 能力涌现
GPT-3 (2020)	175B	600 GB	Few-shot in-context learning

表 1: GPT 系列模型的演进

2.2 Zero-shot Learning

Zero-shot learning 指的是模型在**没有看到任何任务示例**的情况下，直接完成任务。这需要我们巧妙地将任务表述为文本补全问题。

核心思路：既然模型会补全文本，我们就把任务**伪装成**文本补全。例如：

- **问答**：将上下文和问题拼接起来，让模型补全答案
- **摘要**：在文章末尾加上“TL;DR”，模型自然会生成摘要
- **消歧**：将两个候选替换分别填入句子，比较模型赋予的对数概率

GPT-2 的 Zero-shot 能力

GPT-2 无需任何任务特定的微调，仅靠预训练就能在多个基准任务上达到有竞争力的表现。这是一个标志性的结果——意味着足够大的预训练模型已经“内化”了大量任务的解法。

2.3 Few-shot In-context Learning

GPT-3 带来了更令人惊叹的能力：**Few-shot in-context learning**。只需要在 prompt 中提供几个示例，模型就能“学会”新任务——完全**不需要梯度更新**。

In-context Learning 与传统 Fine-tuning 的区别

传统的 fine-tuning 需要：(1) 收集大量标注数据，(2) 对模型进行多轮梯度更新，(3) 可能损害其他任务的性能。而 in-context learning 仅需在推理时的输入中放入几个示例，模型参数完全不变。这种能力随模型规模增大而**涌现**——小模型几乎没有这种能力，但 175B 参数的 GPT-3 表现出了惊人的 few-shot 性能。

2.4 Chain-of-Thought 提示

即使有了 few-shot 能力，模型在复杂推理任务上仍然表现不佳。**Chain-of-Thought (CoT)** 提示技术通过让模型“展示推理过程”来大幅提升性能。

核心思想有两种形式：

1. **Few-shot CoT**：在示例中不仅给出答案，还给出推理过程。模型在回答新问题时也会模仿这种推理链。

2. **Zero-shot CoT**：无需任何示例，仅在 prompt 中添加 “Let’s think step by step”，模型就会自动展开推理过程。

提示工程的本质

提示工程的本质是**理解预训练数据的分布**。当你设计一个 prompt 时，你实际上是在寻找：互联网上什么样的文本之后会跟着我想要的那种回答？“Let’s think step by step”之所以有效，是因为互联网上以这种方式开头的文本通常后面跟着详细的推理过程。然而，这种“欺骗模型”的方式本质上是不令人满意的——我们不应该需要技巧来让模型做正确的事。

2.5 In-context Learning 的局限性

尽管提示技术取得了巨大成功，但存在根本性的局限：

- **上下文窗口有限**：能放入的示例数量受限（虽然现代模型的上下文越来越长）
- **需要“技巧”**：用户需要精心设计 prompt 才能获得好的结果，这对非专业用户不友好
- **无法处理复杂任务**：多步推理、创意写作等任务难以仅通过提示解决
- **模型不是助手**：预训练模型的目标是预测文本，而非帮助用户

这个例子清楚地展示了问题所在：预训练模型不是在**回答**你的问题，而是在**补全**一段看起来合理的文本。这种“不与用户意图对齐”的问题，正是后续方法要解决的核心挑战。

2.6 本章小结

- Zero-shot 和 Few-shot 能力随模型规模增大而涌现，是预训练规模化的重要成果
- Chain-of-Thought 提示通过引导模型“思考”来提升推理能力
- 但提示技术本质上是在“欺骗”模型，用户体验差且能力有限
- 我们需要更根本的方法让模型从“文本补全器”变为“用户助手”

3 指令微调 (Supervised Fine-Tuning)

3.1 从预训练到指令遵循

指令微调 (Instruction Fine-tuning, 也称 Supervised Fine-tuning / SFT) 的核心思想非常直接：收集大量的**(指令, 期望输出)**对，然后用标准的 next-token prediction 损失来微调预训练模型。

指令微调的核心流程

1. **收集数据**：从多种任务（QA、翻译、摘要、代码、推理等）收集指令和期望输出的配对
2. **微调模型**：用这些配对数据对预训练模型进行 supervised fine-tuning
3. **评估**：在未见过的任务上测试模型的泛化能力

这里的关键创新不是技术本身——它只是标准的微调。关键在于**数据的规模和多样性**：不再是针对单一任务微调，而是同时在成千上万种不同任务上微调。

3.2 规模效应：更大的模型从指令微调中获益更多

一个令人鼓舞的趋势是：模型越大，从指令微调中获得的提升就越大。

以 FLAN-T5 系列为例：

- T5-Small (60M 参数)：指令微调提升 +6.1 分
- T5-XXL (11B 参数)：指令微调提升 +**26.6** 分

这意味着预训练的“基础能力”和指令微调的“对齐能力”之间存在正向的协同效应。

3.3 评估：MMLU 基准

MMLU (Massive Multitask Language Understanding) 是一个包含 57 个学科的多选题基准。90% 被视为“人类水平”的标准线。从 GPT-2 时代到 Gemini，模型在此基准上取得了巨大进步。

评估的陷阱

大规模评估面临一个根本性问题：**测试集泄露**。当预训练数据包含数万亿 token 时，很难保证测试题目没有出现在训练数据中。此外，当模型错误时，往往是因为题目本身含糊不清。这使得基准分数的含义变得模糊——90% 以上的得分未必代表真正的“人类水平理解”。

3.4 指令微调的实用经验

近期研究揭示了几个重要的实用发现：

1. **模型蒸馏**：可以使用强大的模型（如 GPT-4）生成指令微调数据，用于训练较小的开源模型。这大大降低了数据收集成本。
2. **数据质量 vs 数量**：“LIMA: Less Is More for Alignment” 论文表明，仅用约 1000 条高质量样本就能达到不错的指令遵循能力。数据质量可能比数量更重要。
3. **众包数据**：Open Assistant 等项目展示了社区驱动的数据收集方式的可行性。

代码数据的特殊价值

课堂讨论中提到一个有趣的观点：代码数据天然具有“逐步推理”的结构——每一行代码都是将问题分解为更小步骤的过程。因此，在预训练混合数据中增加代码数据的比例，已被证明能提升模型的推理能力，即使在非代码任务上也是如此。

3.5 指令微调的三大局限

指令微调虽然有效，但存在三个根本性局限，正是这些局限推动了 RLHF 和 DPO 的发展：

局限 1：数据收集昂贵。预训练可以直接爬取互联网数据，但指令微调需要人工编写高质量的回答。随着任务复杂度提升（如物理学博士级别的问题），这变得越来越昂贵。

局限 2：开放性任务无标准答案。对于创意写作、故事生成等任务，不存在唯一的“正确答案”。这使得收集训练数据变得困难——你该让标注者写什么样的回答？

局限 3：Token 级损失无法区分错误严重程度。Next-token prediction 损失函数对所有 token 级错误一视同仁。但将“Avatar”描述为“adventure show”（可以接受）和描述为“musical”（严重错误）的代价是完全不同的。

更深层的问题：目标函数不匹配

指令微调最根本的问题是：我们的真正目标是让模型生成人类喜欢的输出，但优化目标仍然是 token 级的对数似然。这两者之间存在本质的不匹配。此外，随着模型变得越来越强大，人类标注者的回答质量可能反而成为瓶颈——模型生成的回答可能已经比人类标注者的更好。

3.6 本章小结

- 指令微调通过在大量（指令，回答）对上微调预训练模型，使其学会遵循指令
- 更大的模型从指令微调中获益更多，体现了预训练和对齐之间的正向协同
- 数据质量可能比数量更重要；可以用强大模型生成训练数据
- 三大局限——高昂的数据收集成本、开放任务缺乏标准答案、损失函数无法区分错误严重程度——推动了基于偏好的优化方法的发展

4 基于人类反馈的强化学习（RLHF）

4.1 核心动机：直接优化人类偏好

指令微调仍然在做 token 级的预测任务，而我们真正想做的是**直接优化人类偏好**。RLHF（Reinforcement Learning from Human Feedback）正是为此而生。

RLHF 的完整流程

1. **Step 1: 指令微调（SFT）**——从预训练模型出发，用指令-回答对微调，得到一个初始的助手模型
2. **Step 2: 训练奖励模型**——收集人类偏好数据（哪个回答更好），训练一个预测人类偏好的奖励模型
3. **Step 3: RL 优化**——用强化学习算法（如 PPO）优化语言模型，使其生成奖励模型评分更高的回答

4.2 问题形式化

假设我们有一个**奖励函数** $r(x, y)$ ，它能评价对于输入 x ，回答 y 的质量。我们的目标是：

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim p_{\theta}(\cdot|x)} [r(x, y)]$$

其中：

- $p_{\theta}(y|x)$ ：我们正在优化的语言模型
- $r(x, y)$ ：奖励函数
- $\mathcal{D}$ ：指令分布

与之前方法的根本区别

注意目标函数中 $y \sim p_{\theta}(\cdot|x)$ ——我们是从**模型自身**采样的！这与预训练和 SFT 截然不同：

- 预训练/SFT：数据来自外部（互联网/人类标注），优化 token 级对数似然
- RLHF：数据来自模型自身的采样，优化一个（可能不可微的）奖励函数

这种 “on-policy” 的训练方式使得模型可以探索并学会生成比训练数据更好的回答。

4.3 Step 2: 训练奖励模型

4.3.1 为什么需要奖励模型？

直接让人类为每个模型输出打分是不可行的——我们需要在数百万条回答上评估，而人类标注的速度远远不够。解决方案：训练一个**奖励模型**来**预测**人类会给出什么评价。

4.3.2 为什么用排序而非评分？

让人类给一个回答打绝对分数（如 1-10 分）是**极其不可靠**的：

- 不同人对同一回答可能给出完全不同的分数（4.1 vs 6.6）
- 同一个人不同日子给出的分数也会不同
- 没有统一的评分标准可以消除这种主观性

但让人类**比较两个回答哪个更好**则容易得多，且一致性更高。这就是为什么 RLHF 使用**偏好排序数据**而非绝对评分。

4.3.3 Bradley-Terry 模型

如何从排序数据中学到一个能输出分数的奖励模型？**Bradley-Terry 模型**提供了数学框架。它假设人类选择回答 y_1 优于 y_2 的概率为：

$$p(y_1 \succ y_2 | x) = \sigma(r(x, y_1) - r(x, y_2))$$

其中 σ 是 sigmoid 函数。直觉很清晰：两个回答的奖励**差值**越大，人类选择较好回答的概率越高。

奖励模型的训练目标就是**最大化**以下对数似然：

$$\mathcal{L}_{\text{RM}}(\phi) = \mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))]$$

其中 y_w 是人类偏好的“赢”的回答， y_l 是“输”的回答。

奖励模型的实现

奖励模型通常直接从**预训练语言模型**初始化，然后在其输出端添加一个标量头（scalar head）来预测分数。这样做的好处是奖励模型能够理解文本的语义，从而更准确地预测人类偏好。输入 (x, y) 被拼接后送入模型，模型输出一个标量分数。

4.4 Step 3: RL 优化与 KL 约束

有了奖励模型后，我们可以优化语言模型。但直接最大化奖励存在严重的**奖励模型黑客**（Reward Hacking）问题。

Reward Hacking：优化学到的指标的危險

奖励模型是一个**学到的近似**，它不可避免地存在误差。当我们过度优化这个近似的奖励模型时，语言模型会学会利用奖励模型的“漏洞”——生成一些奖励模型给高分但实际质量很差的输出（例如重复的短语、无意义的文本）。这是一个普遍的机器学习原则：**永远不要无限度地优化一个学到的指标**。

解决方案是添加 **KL 散度约束**，防止优化后的模型偏离初始模型太远：

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim p_{\theta}(\cdot|x)} \left[r_{\phi}(x, y) - \beta \log \frac{p_{\theta}(y|x)}{p_{\text{ref}}(y|x)} \right]$$

其中：

- $r_{\phi}(x, y)$ ：学到的奖励模型
- $p_{\text{ref}}(y|x)$ ：参考模型（通常是 SFT 模型）
- β ：控制约束强度的超参数
- $\beta \log \frac{p_{\theta}(y|x)}{p_{\text{ref}}(y|x)}$ ：KL 散度惩罚项

KL 约束的两层含义

KL 约束不仅仅是一个正则化技巧，它有深刻的动机：

1. **防止 reward hacking**：限制模型不能偏离初始分布太远，因为奖励模型只在初始分布附近是可靠的
2. **保持语言能力**：初始模型是一个好的语言模型，过度优化可能破坏其语言生成能力

4.5 RLHF 的惊人效果

RLHF 的一个标志性结果是：即使是**很小的模型**，经过 RLHF 训练后，其生成的摘要也能被人类评为**优于人类撰写的摘要**。这是 SFT 做不到的——SFT 的性能受限于训练数据的质量（即人类标注者的水平），而 RLHF 可以**超越**数据中的人类水平。

RLHF 为什么能超越人类数据？

在 SFT 中，模型试图模仿人类标注者的回答，因此其上限是标注者的水平。而在 RLHF 中，人类只需要**判断哪个更好**（这比**写出好答案**容易得多），模型则通过探索和优化来发现比现有回答更好的生成方式。这就是为什么 RLHF 能让模型超越人类水平的关键所在。

4.6 RLHF 的复杂性

尽管效果显著，RLHF 的实现极其复杂：

- 需要同时维护**四个**模型：策略模型、参考模型、奖励模型、价值函数
- PPO 算法对超参数极其敏感
- 需要大量的采样和计算
- 训练不稳定，调试困难
- 长期以来仅限于计算资源充足的大公司使用

这些问题促使研究者寻找更简单的替代方案——这就是 DPO 的动机。

4.7 本章小结

- RLHF 通过直接优化人类偏好来对齐语言模型，克服了 SFT 的目标函数不匹配问题
- 使用偏好排序而非绝对评分来收集人类反馈，通过 Bradley-Terry 模型转化为奖励分数
- KL 约束防止模型过度优化学到的奖励模型（reward hacking）
- RLHF 能让模型超越人类数据的质量上限
- 但实现极其复杂，推动了 DPO 等更简单方法的发展

5 直接偏好优化（DPO）

5.1 核心思想：用语言模型自身表达奖励

DPO（Direct Preference Optimization）的核心洞察是：**奖励模型可以用语言模型自身来表达**，从而完全绕过强化学习步骤。

直觉上理解：一个好的语言模型应该把更高的概率分配给人类喜欢的回答。因此，模型的对数概率**本身**就可以作为一种隐式的奖励信号。

DPO 的思路

1. RLHF 管线中，唯一的外部信息来源是**人类偏好标注**
2. 奖励模型只是偏好数据的一个中间表示
3. 如果能直接用偏好数据优化语言模型参数，就可以**同时省去**奖励模型训练和 RL 优化两个步骤
4. 最终问题变成一个简单的**二分类**问题

5.2 数学推导

5.2.1 Step 1: KL 约束优化的闭式解

回忆 RLHF 的优化目标：

$$\max_{\theta} \mathbb{E}_{y \sim p_{\theta}(\cdot|x)} \left[r(x, y) - \beta \log \frac{p_{\theta}(y|x)}{p_{\text{ref}}(y|x)} \right]$$

这个优化问题有**闭式解**（类似于 Boltzmann 分布）：

$$p^*(y|x) = \frac{1}{Z(x)} p_{\text{ref}}(y|x) \cdot \exp \left(\frac{r(x, y)}{\beta} \right)$$

其中 $Z(x) = \sum_y p_{\text{ref}}(y|x) \cdot \exp \left(\frac{r(x, y)}{\beta} \right)$ 是归一化常数（配分函数）。

直觉：最优策略就是将参考模型的分布**按奖励的指数加权**——高奖励的补全获得更高概率，低奖励的获得更低概率。 β 控制这种重新加权的程度： β 越小，越偏向高奖励； β 越大，越接近参考分布。

5.2.2 Step 2: 用语言模型表达奖励

对闭式解取对数并移项，可以将奖励表达为语言模型参数的函数：

$$r(x, y) = \beta \log \frac{p_{\theta}(y|x)}{p_{\text{ref}}(y|x)} + \beta \log Z(x)$$

这个等式告诉我们：一个补全 y 的**隐式奖励**由两部分决定：

- $\beta \log \frac{p_{\theta}(y|x)}{p_{\text{ref}}(y|x)}$ ：当前模型相对于参考模型赋予 y 的“额外概率”
- $\beta \log Z(x)$ ：一个只依赖于 x 的归一化常数

“每个策略都隐式定义了一个奖励模型”

这是一个深刻的洞察：不只是最优策略对应某个奖励模型，**任何策略 p_{θ} 都隐式地定义了某个奖励函数**。这种一一对应关系使得我们可以直接通过优化策略来优化奖励，反之亦然。

5.2.3 Step 3: 配分函数的消除

将上述奖励表达式代入 Bradley-Terry 模型：

$$p(y_w \succ y_l|x) = \sigma(r(x, y_w) - r(x, y_l))$$

由于 $r(x, y_w) - r(x, y_l)$ 中，**配分函数 $\beta \log Z(x)$ 被消去**（因为两个奖励共享同一个 x ， $Z(x)$ 在两项中相同），我们得到：

$$p(y_w \succ y_l|x) = \sigma \left(\beta \log \frac{p_{\theta}(y_w|x)}{p_{\text{ref}}(y_w|x)} - \beta \log \frac{p_{\theta}(y_l|x)}{p_{\text{ref}}(y_l|x)} \right)$$

5.2.4 最终的 DPO 损失函数

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{p_{\theta}(y_w|x)}{p_{\text{ref}}(y_w|x)} - \beta \log \frac{p_{\theta}(y_l|x)}{p_{\text{ref}}(y_l|x)} \right) \right]$$

DPO 损失函数的直觉解读

DPO 损失的含义极其清晰：

- 对于人类偏好的“赢”的回答 y_w ，**增大**其相对于参考模型的概率比
- 对于“输”的回答 y_l ，**减小**其相对于参考模型的概率比
- 这本质上就是一个**二分类问题**：输入是两个回答的对数概率比之差，标签是人类偏好
- 不需要奖励模型，不需要采样，不需要 RL——只需要标准的梯度下降！

5.3 DPO 的实验效果

实验表明，DPO 在多个任务上达到了与完整 RLHF (PPO) **相当的性能**，但实现简单得多。在摘要任务上，DPO 和 PPO 的表现几乎相同。

5.4 DPO 的广泛采用

DPO 的简洁性使其迅速被开源社区和工业界广泛采用：

- HuggingFace Open LLM Leaderboard 上排名靠前的模型中，9/10 使用了 DPO
- Mistral、Llama 3 等主要开源模型都采用了 DPO
- DPO 使得小团队和研究者也能进行偏好对齐，大大降低了门槛

5.5 SFT vs RLHF/DPO 的输出对比

经过 RLHF/DPO 优化的模型，回答通常具有以下特征（与纯 SFT 模型相比）：

- 更详细和全面的内容
- 更好的组织结构（使用列表、分段等）
- 更符合人类阅读习惯的表达方式
- 语气更加自信和权威

5.6 本章小结

- DPO 通过一个优雅的数学推导，将 RLHF 的三步流程简化为**一步二分类优化**
- 核心洞察：语言模型的对数概率隐式定义了一个奖励模型，配分函数在偏好差值中被消去
- DPO 的性能与完整 RLHF 管线相当，但实现简单得多
- 已被工业界和开源社区广泛采用（Mistral、Llama 3 等）

6 从 InstructGPT 到 ChatGPT

6.1 InstructGPT: 定义管线

InstructGPT 是第一个系统性地将前述所有技术组合在一起的模型，它定义了现代 LLM 后训练的标准管线。

InstructGPT 的关键数据：

- 约 **30,000 种不同任务** 的指令微调数据
- 使用人类标注者收集偏好排序
- 通过 PPO 进行 RLHF 优化

效果对比非常直观：GPT-3 面对“向 6 岁小孩解释登月”的问题，会继续生成更多问题（因为它只是在补全文本）；而 InstructGPT 则直接给出适合 6 岁儿童理解的简明解释。

6.2 ChatGPT: 对话优化

ChatGPT 在 InstructGPT 的基础上，进一步针对**对话交互**进行了优化：

核心技术不变 (SFT + RLHF)，关键差异在于数据——更多的对话格式数据、更注重多轮交互的连贯性。

6.3 本章小结

InstructGPT 和 ChatGPT 证明了后训练管线的有效性：通过 SFT 建立指令遵循能力，通过 RLHF/DPO 进一步对齐人类偏好。这个管线已经成为现代 LLM 开发的标准流程。

7 开放挑战与未来方向

7.1 Reward Hacking

Reward hacking 是 RLHF 面临的最严重的技术挑战之一。强化学习是一个极其强大的优化算法（AlphaGo 和 AlphaZero 就是例证），当它被用来优化一个不完美的奖励模型时，它会找到各种“取巧”的方式来获取高分，而不是真正地完成我们期望的任务。

7.2 冗长性问题 (Verbosity)

一个普遍观察到的现象是：经过 RLHF 训练的模型倾向于生成**过于冗长**的回答。原因令人深思：

人类偏好 $\neq$ 人类利益

在大规模数据标注中，标注者 (turkers) 倾向于选择**更长**的回答作为“更好”的——因为他们可能没有仔细阅读内容，而是用长度作为质量的代理指标。这种偏差通过 RLHF 被放大：模型学到了“更长 = 更好”的错误信号。更广泛地说，人类偏好并不总是等同于人类的最佳利益——人类可能偏好听起来权威但不正确的回答，而非谦逊但准确的回答。

7.3 幻觉问题

幻觉 (Hallucination) 是 LLM 的顽疾, RLHF 并不能解决这个问题。事实上, RLHF 可能使幻觉更难被发现——因为模型学会了用更自信的语气表达不正确的信息。

7.4 本章小结

- Reward hacking 是 RLHF 最严重的技术挑战, 需要精心设计奖励模型和约束
- 人类偏好数据的质量直接影响模型行为——标注者的偏差会被放大
- 冗长性和幻觉等问题仍然是活跃的研究领域
- 如何设计更好的评估方法和更可靠的对齐技术, 是未来的核心问题

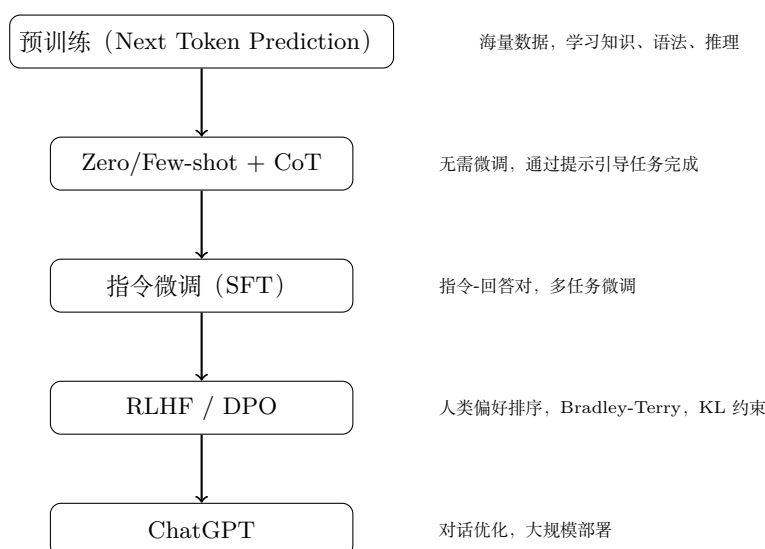
8 总结与延伸

8.1 讲者的核心总结

Archit Sharma 在课程结尾总结了从预训练到 ChatGPT 的完整路径:

1. **预训练**: 在海量文本上训练 next-token prediction, 获得强大的基础能力
2. **Zero/Few-shot**: 发现预训练模型具有涌现的 in-context learning 能力
3. **SFT**: 通过指令-回答对微调, 使模型学会遵循指令
4. **RLHF/DPO**: 通过人类偏好数据直接优化模型行为, 超越人类标注数据的质量上限

8.2 全课知识图谱



8.3 关键 Takeaways

五条核心原则

1. **预训练是基础**：几乎所有后训练的效果都依赖于强大的预训练基座。模型越大、预训练越充分，后训练的收益越大
2. **数据质量重于数量**：无论是 SFT 还是偏好数据，高质量的少量数据可能优于低质量的大量数据（“Less Is More for Alignment”）
3. **比较比评分容易**：人类更擅长比较两个回答的优劣，而非给单个回答打绝对分数。这一洞察是 RLHF 和 DPO 的数据基础
4. **DPO 是 RLHF 的优雅替代**：通过巧妙的数学推导，DPO 将复杂的 RL 优化简化为简单的二分类损失，且效果相当
5. **对齐不等于完美**：Reward hacking、冗长性、幻觉等问题提醒我们，“优化人类偏好”并不等同于“做正确的事”

8.4 拓展阅读

- Radford et al., 2018. *Improving Language Understanding by Generative Pre-Training* (GPT-1)
- Radford et al., 2019. *Language Models are Unsupervised Multitask Learners* (GPT-2)
- Brown et al., 2020. *Language Models are Few-Shot Learners* (GPT-3): <https://arxiv.org/abs/2005.14165>
- Wei et al., 2022. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*: <https://arxiv.org/abs/2201.11903>
- Kojima et al., 2022. *Large Language Models are Zero-Shot Reasoners*: <https://arxiv.org/abs/2205.11916>
- Ouyang et al., 2022. *Training language models to follow instructions with human feedback* (InstructGPT): <https://arxiv.org/abs/2203.02155>
- Stiennon et al., 2020. *Learning to summarize from human feedback*: <https://arxiv.org/abs/2009.01325>
- Rafailov et al., 2023. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model* (DPO): <https://arxiv.org/abs/2305.18290>
- Zhou et al., 2023. *LIMA: Less Is More for Alignment*: <https://arxiv.org/abs/2305.11206>
- Christiano et al., 2017. *Deep Reinforcement Learning from Human Preferences*: <https://arxiv.org/abs/1706.03741>